

Γραφική με Υπολογιστές 2026

Εργασία #1: Πλήρωση Τριγώνων

Ζητούμενα

Συνάρτηση γραμμικής παρεμβολής

Να δημιουργηθεί μία συνάρτηση η οποία υπολογίζει την τιμή V ενός διανύσματος στη θέση $p = (x, y)$ μέσω γραμμικής παρεμβολής μεταξύ των διανυσμάτων V_1 και V_2 που αντιστοιχούν στις θέσεις $p_1 = (x_1, y_1)$ και $p_2 = (x_2, y_2)$ με την υπόθεση ότι το σημείο p ανήκει στο ευθύγραμμο τμήμα p_1p_2 .

$$V = \text{vector_interp}(p_1, p_2, V_1, V_2, \text{coord}, \text{dim})$$

όπου:

- p_1 και p_2 οι δισδιάστατες συντεταγμένες των σημείων στα οποία αντιστοιχούν τα V_1, V_2 .
- V_1 και V_2 οι διανυσματικές τιμές στις θέσεις p_1 και p_2 .
- coord η τετμημένη (x) ή η τεταγμένη (y) του σημείου p ανάλογα με το αν $\text{dim} = 1$ ή $\text{dim} = 2$ αντίστοιχα.

Η συνάρτηση αυτή θα σας είναι χρήσιμη και σε επόμενες εργασίες.

Συναρτήσεις Πλήρωσης Τριγώνων

Σκοπός της εργασίας είναι η υλοποίηση ενός αλγορίθμου πλήρωσης τριγώνων με βάση τον αλγόριθμο πλήρωσης πολυγώνων που περιγράφεται στις σημειώσεις. Αφού προσαρμόσετε κατάλληλα τον αλγόριθμο των σημειώσεων ώστε να χειρίζεται την ειδική περίπτωση των τριγώνων εξετάστε τις δύο διαφορετικές εκδοχές απόδοσης χρώματος που περιγράφονται στη συνέχεια.

Flat shading

Έστω τρίγωνο που ορίζεται από κορυφές με ακέραιες συντεταγμένες. Υποθέστε καμβά διάστασης $M \times N$. Δημιουργήστε τη συνάρτηση,

$$\text{updated_img} = f_shading(\text{img}, \text{vertices}, \text{vcolors})$$

Όπου:

- img είναι η εικόνα (πίνακας διάστασης $M \times N \times 3$) με τυχόν προϋπάρχοντα τρίγωνα.
- $vertices$ είναι ακέραιος πίνακας διάστασης 3×2 που σε κάθε γραμμή περιέχει τις δισδιάστατες συντεταγμένες μιας κορυφής του τριγώνου.
- $vcolors$ είναι πίνακας διάστασης 3×3 που σε κάθε γραμμή του περιέχει το χρώμα της αντίστοιχης κορυφής του τριγώνου σε μορφή RGB (με τιμές στο διάστημα $[0, 1]$).
- $updated_img$ είναι πίνακας διάστασης $M \times N \times 3$ που για όλα τα σημεία του τριγώνου περιέχει τις υπολογισμένες χρωματικές συνιστώσες (R_i, G_i, B_i) καθώς και τα προϋπάρχοντα τρίγωνα της εισόδου img (επικαλύπτοντας τυχόν κοινά χρωματισμένα σημεία που προϋπήρχαν από την πλήρωση άλλων τριγώνων).

Όλα τα pixels του τριγώνου θα βάφονται με χρώμα ίσο προς τον διανυσματικό μέσο όρο των χρωμάτων των τριών κορυφών.

Gouraud shading

Έστω τρίγωνο που ορίζεται από κορυφές με ακέραιες συντεταγμένες. Υποθέστε καμβά διάστασης $M \times N$. Δημιουργήστε τη συνάρτηση,

$$updated_img = g_shading(img, vertices, vcolors)$$

Όπου:

- img είναι η εικόνα (πίνακας διάστασης $M \times N \times 3$) με τυχόν προϋπάρχοντα τρίγωνα.
- $vertices$ είναι ακέραιος πίνακας διάστασης 3×2 που σε κάθε γραμμή περιέχει τις δισδιάστατες συντεταγμένες μιας κορυφής του τριγώνου.
- $vcolors$ είναι πίνακας διάστασης 3×3 που σε κάθε γραμμή του περιέχει το χρώμα της αντίστοιχης κορυφής του τριγώνου σε μορφή RGB (με τιμές στο διάστημα $[0, 1]$).
- $updated_img$ είναι πίνακας διάστασης $M \times N \times 3$ που για όλα τα σημεία του τριγώνου περιέχει τις υπολογισμένες χρωματικές συνιστώσες (R_i, G_i, B_i) καθώς και τα προϋπάρχοντα τρίγωνα της εισόδου img (επικαλύπτοντας τυχόν κοινά χρωματισμένα σημεία που προϋπήρχαν από την πλήρωση άλλων τριγώνων).

Όλα τα pixels του τριγώνου θα βάφονται μέσω γραμμικής παρεμβολής των χρωμάτων των τριών κορυφών του τριγώνου.

Shading με texture maps

Δημιουργήστε τη συνάρτηση,

$$updated_img = t_shading(img, vertices, uv, textImg)$$

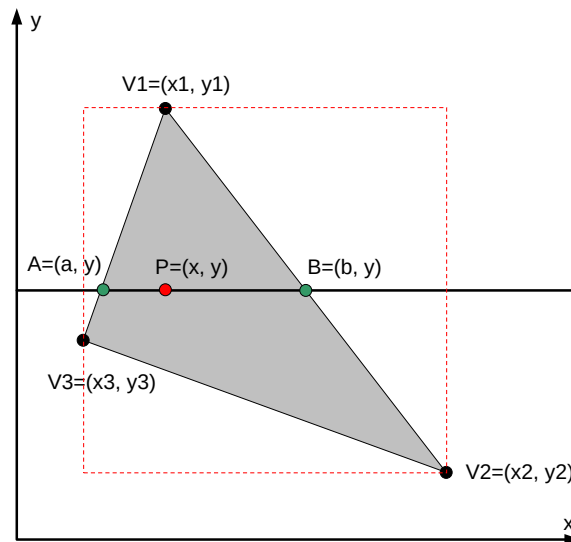
Όπου:

- img είναι η εικόνα (πίνακας διάστασης $M \times N \times 3$) με τυχόν προϋπάρχοντα τρίγωνα.

- *vertices* είναι ακέραιος πίνακας διάστασης 3×2 που σε κάθε γραμμή περιέχει τις διαστάσεις συντεταγμένες μιας κορυφής του τριγώνου.
- *uv* είναι πίνακας διάστασης 3×2 που σε κάθε γραμμή του περιέχει τις κανονικοποιημένες συντεταγμένες $(u_i, v_i) \in [0, 1]^2$, $i = 1, 2, 3$ που δείχνουν σε 3 θέσεις της εικόνας *texture textImg*.
- *textImg* είναι εικόνα texture (πίνακας οιάστασης $K \times L \times 3$) που περιέχει εικόνα που θα αποτυπωθεί στο αντικείμενο. Συγκεκριμένα στο τρέχον τρίγωνο θα αποτυπωθεί το τμήμα της εικόνας *textImg* που αντιστοιχεί σε τριγωνική περιοχή με συντεταγμένες $u_i \times K, v_i \times L$.
- *updated_img* είναι πίνακας διάστασης $M \times N \times 3$ που για όλα τα σημεία του τριγώνου περιέχει τις υπολογισμένες χρωματικές συνιστώσες (R_i, G_i, B_i) καθώς και τα προϋπάρχοντα τρίγωνα της εισόδου *img* (επικαλύπτοντας τυχόν κοινά χρωματισμένα σημεία που προϋπήρχαν από την πλήρωση άλλων τριγώνων).

Κάθε σημείο του τριγώνου θα έχει το χρώμα που θα αντιγράφεται από θέση (x, y) της *textImg* που αντιστοιχεί στις κανονικοποιημένες συντεταγμένες (u, v) οι οποίες θα υπολογίζονται με γραμμική παρεμβολή από τις κανονικοποιημένες συντεταγμένες των κορυφών του. Συγκεκριμένα, για το χρωματισμό του τριγώνου, με αναφορά στα σημεία του σχήματος 1, πρώτα θα υπολογίζονται οι κανονικοποιημένες συντεταγμένες στις θέσεις A και B, με χρήση της συνάρτησης *vector_interp* για τις τιμές των κανονικοποιημένων συντεταγμένων των κορυφών C1, C3 και C1, C2 αντίστοιχα (δίνοντας τις κατάλληλες συνιστώσες των συντεταγμένων των κορυφών του τριγώνου). Η πρώτη αυτή φάση θα υλοποιείται μία φορά για κάθε scanline *y*. Σε δεύτερη φάση, θα πρέπει πάλι με την χρήση της *vector_interp* να γίνει γραμμική παρεμβολή για κάθε σημείο $P = (x, y)$ που ανήκει στο τρέχον scanline.

Οποτεδήποτε χρειάζεται να αντιγραφεί χρώμα από μη ακέραια θέση (x, y) του *textImg* θα χρησιμοποιείται μια τοπική παρεμβολή. Στην απλούστερη περίπτωση θα επιλέγεται η τιμή της πλησιέστερης ακέραιας θέσης.



Σχήμα 1: Παράδειγμα χρωματισμού

Συνάρτηση χρωματισμού αντικειμένου

Να υλοποιήσετε τη συνάρτηση:

$img = render_img(faces, vertices, vcolors, uvs, depth, shading, texImg)$

Όπου:

- img είναι έγχρωμη εικόνα διάστασης $M \times N \times 3$. Η εικόνα θα περιέχει K χρωματισμένα τρίγωνα τα οποία σχηματίζουν την προβολή ενός 3D αντικειμένου στις 2 διαστάσεις.
- $faces$ είναι ο πίνακας που περιέχει τις κορυφές των K τριγώνων. Ο πίνακας είναι διάστασης $K \times 3$. Η i -στη γραμμή του πίνακα δηλώνει τις τρεις κορυφές που σχηματίζουν το τρίγωνο (με αναφορά σε κορυφές του πίνακα $vertices$ και αρίθμηση που ξεκινά από το 0).
- $vertices$ είναι ο πίνακας με τις κορυφές των τριγώνων της εικόνας. Ο πίνακας $vertices$ είναι διάστασης $L \times 2$ και περιέχει τις συντεταγμένες ενός πλήθους L κορυφών. Για απλούστευση υποθέστε ότι όλες βρίσκονται εντός του καμβά.
- $vcolors$ είναι ο πίνακας με τα χρώματα των κορυφών. Ο πίνακας $vcolors$ είναι διάστασης $L \times 3$. Η i -στη γραμμή του πίνακα δηλώνει τις χρωματικές συνιστώσες της αντίστοιχης κορυφής.
- uvs είναι πίνακας διάστασης $L \times 2$ που σε κάθε γραμμή του περιέχει τις κανονικοποιημένες συντεταγμένες $(u, v) \in [0, 1]^2$, που δείχνουν τις θέσεις της εικόνας texture $texImg$.
- $depth$ είναι ο πίνακας που δηλώνει το βάθος της κάθε κορυφής πριν την προβολή του αντικειμένου στις 2 διαστάσεις. Ο πίνακας $depth$ είναι διάστασης $L \times 1$.
- $shading$ είναι παρόμοια μεταβλητή ελέγχου (τύπου string) που καθορίζει τη συνάρτηση χρωματισμού ("flat", "gouraud", ή "tex") που θα χρησιμοποιηθεί, και μπορεί να πάρει τιμές "f" ή "t" αντίστοιχα.
- M και N είναι το ύψος και το πλάτος του καμβά αντίστοιχα.

Στο εσωτερικό της συνάρτησης $render_img$ ανάλογα με την τιμή της μεταβλητής $shading$, θα καλείται η ανάλογη ρουτίνα για το χρωματισμό των εσωτερικών σημείων κάθε τριγώνου. **Η σειρά με την οποία πρέπει να χρωματιστούν τα τρίγωνα προκύπτει από τον πίνακα βάθους $depth$.** Ο χρωματισμός θα πρέπει να ξεκινάει από τα μακρινότερα (αυτά με το μεγαλύτερο βάθος) τρίγωνα και να συνεχίζει με τα κοντινότερα. Το βάθος ενός τριγώνου υπολογίζεται ως το κέντρο βάρους τους βάθους των κορυφών του.

Θεωρήστε δεδομένο ότι:

- Το background του καμβά είναι λευκό ($rgb = (1,1,1)$).
- Ο καμβάς έχει διαστάσεις $M = 512$, $N = 512$.
- Προσπαθήστε όπου είναι δυνατό στον κώδικα, σας να χρησιμοποιήσετε vectorization προκειμένου να αποφύγετε υπερβολικά μεγάλους χρόνους εκτέλεσης (ο τρόπος για να το κάνετε αυτό στη βιβλιοθήκη numpy, βλ. 1, είναι παρόμοιος με το matlab).

Παραδοτέα

1. Οι παραπάνω συναρτήσεις σε μορφή **σχολιασμένου** πηγαίου κώδικα Python ($\geq v3.10$) με σχόλια γραμμένα στα **αγγλικά** ή **greeklish**. (κοινώς, **μη γράφετε σχόλια με ελληνικούς χαρακτήρες**).
2. **Τρία** scripts επίδειξης με ονόματα `demo_f`, `demo_g.py`, `demo_t.py`, τα οποία θα καλούνται χωρίς εξωτερικά ορίσματα και θα:
 - (α') Φορτώνουν τα δεδομένα εισόδου (πίνακες `v_pos`, `v_clr`, `t_pos_idx`, `depth` της `render_img`) από το αρχείο `hw1.npy` που σας δίνεται.
 - (β') Πραγματοποιούν τον χρωματισμό των τριγώνων του αντικειμένου χρησιμοποιώντας τη συνάρτηση `render_img` (με την επιλογή του αντιστοιχού τύπου χρωματισμού, "flat", "gouraud", ή "texture").
 - (γ') Αποθηκεύουν την τελική εικόνα (για τον τρόπο αποθήκευσης δείτε το Υπόμνημα).
3. Αναφορά με:
 - Περιγραφή της λειτουργίας και του τρόπου κλήσης των προγραμμάτων,
 - Περιγραφή της διαδικασίας χρωματισμού των τριγώνων που ακολουθήσατε και παρουσίαση του αντίστοιχου ψευδοκώδικα
 - Περιγραφή των παραδοχών που χρησιμοποιήσατε.
 - Τα ενδεικτικά αποτελέσματα που παράγονται από τα demos.

Υποβολή εργασίας

- Οι εργασίες είναι **αυστηρά** ατομικές.
- Υποβάλετε ένα και μόνο αρχείο, τύπου `zip`.
- Το όνομα του αρχείου πρέπει να είναι `AEM.zip`, όπου `AEM` είναι τα τέσσερα ψηφία του Α.Ε.Μ. σας.
- Το προς υποβολή αρχείο πρέπει να περιέχει τα αρχεία κώδικα python (`version = 3.10`) και το αρχείο `report.pdf` το οποίο θα είναι η αναφορά της εργασίας.
- Η αναφορά πρέπει να είναι ένα αρχείο τύπου `PDF`, και να έχει όνομα `report.pdf`.
- Όλα τα αρχεία κώδικα πρέπει να είναι αρχεία κειμένου τύπου `UTF-8`, και να έχουν κατάληξη `.py`.
- Το αρχείο τύπου `zip` που θα υποβάλετε **ΔΕΝ** πρέπει να περιέχει κανέναν εσωτερικό υποφάκελο.
- Για την ονομασία των αρχείων που περιέχονται στο προς υποβολή αρχείο, χρησιμοποιείτε μόνο αγγλικούς χαρακτήρες, και όχι ελληνικούς ή άλλα σύμβολα, πχ "#", "\$", "%" κλπ.

Προσοχή: Θα αξιολογηθούν μόνο όσες εργασίες έχουν **demos που τρέχουν!**

1 Υπόμνημα

Σε αυτό το υπόμνημα ακολουθούν οδηγίες για την εγκατάσταση της python, καθώς και τις βιβλιοθήκες που θα σας χρησιμεύσουν για την υλοποίηση της εργασίας.

Εγκατάσταση

Για την εγκατάσταση της python σε περιβάλλον Windows μπορείτε πολύ εύκολα να κατεβάσετε τον αντίστοιχο installer από το επίσημο site της python.

Για την εγκατάσταση της python σε περιβάλλον Linux (και συγκεκριμένα Ubuntu ≥ 22.04) υπάρχει προεγκατεστημένη python v3.10 (22.04) και python v3.12 (24.04).

Code editor

Ένας ευρέως διαδεδομένος code editor (για πολλές γλώσσες προγραμματισμού αλλά ειδικά και για python) είναι το VS Code της Microsoft, ο οποίος πάλι μπορεί να εγκατασταθεί πολύ εύκολα και σε Windows και σε Ubuntu.

Το VS Code δέχεται extensions (τα οποία μπορείτε να εγκαταστήσετε πολύ εύκολα μέσω της ίδιας της εφαρμογής). Για τη λειτουργία του VS Code με Python και για να μπορείτε να χρησιμοποιήσετε debugger, καλό θα ήταν να εγκαταστήσετε το extension για την Python.

Υλοποίηση και βιβλιοθήκες

Για την υλοποίηση της εργασίας θα χρησιμοποιηθεί σε μεγάλο βαθμό η βιβλιοθήκη **numpy**, μία πολύ ευρέως γνωστή βιβλιοθήκη για χρήση και επεξεργασία πινάκων και τανυστών, της οποίας οι δυνατότητες αντιστοιχούν με σχεδόν 1-1 τρόπο με το Matlab. Για να εγκαταστήσετε τη βιβλιοθήκη numpy μπορείτε πολύ εύκολα σε ένα τερματικό να τρέξετε την ακόλουθη εντολή:

```
pip install numpy
```

pip είναι ο package manager της python, και σχεδόν όλες οι βιβλιοθήκες της python μπορούν να εγκατασταθούν με παρόμοιες εντολές.

Ακόμα μία χρήσιμη βιβλιοθήκη είναι η OpenCV (pip install opencv-python), η οποία μπορεί να σας χρησιμεύσει για να διαβάζετε και να αποθηκεύετε εικόνες. Επίσης μία ακόμα βιβλιοθήκη, χρήσιμη για διαδραστική απεικόνιση εικόνων (και με συντακτικό πολύ παρόμοιο με matlab) είναι η matplotlib (pip install matplotlib).

Για οποιαδήποτε απορία σχετικά με την εργασία, την python και τη χρήση της για την υλοποίηση της εργασίας, μη διστάσετε να επικοινωνήσετε με τον κ. Αντώνη Καρακώττα (akarakott@ece.auth.gr)